

Use Case Tracker App

Architecture, access model, and Rocketship integration

STATUS Draft	SERIES Part 4 of 4 - Secure Databricks Apps + MCP documentation series
AUDIENCE Architecture, security, and platform stakeholders	DATE April 2026

EXECUTIVE SUMMARY

The Use Case Tracker App is the structured product interface for capturing, reviewing, and progressing AI and analytics use cases. It gives teams a governed way to manage use-case intake and status while keeping the underlying data and automation model consistent across apps and agents.

Table of Contents

1. Executive Summary
2. How to Read This Document
3. Ecosystem Architecture
4. Product Purpose
5. Access and Permissions
6. Rocketship Integration
7. Source Notes

Documentation Series

This document is part of a four-document series:

1. [MCP Concepts for Governed Apps and Agents](#) - reusable MCP concepts, terminology, and design principles.
2. [Secure Databricks Apps + MCP Reference Blueprint](#) - a generic Databricks reference architecture for secure MCP-based systems.
3. [Use Case Tracker MCP Architecture](#) - the concrete Use Case Tracker implementation of the blueprint.
4. **Use Case Tracker App** - product architecture, access model, and Rocketship integration for the tracker app.

Together, the four documents tell one stream of work: the concept, the reusable Databricks pattern, the Use Case Tracker MCP implementation, and the product-facing app view. Executive readers can skim the executive summaries in order; implementation teams can use the later sections as operating references.

1. Executive Summary

The Use Case Tracker App is the structured product interface for capturing, reviewing, and progressing AI and analytics use cases. It gives teams a governed way to manage use-case intake and status while keeping the underlying data and automation model consistent across apps and agents.

The app sits in a broader ecosystem:

Teams launch the tracker and companion agents from Rocketship

- > Databricks Apps host the product UI and agents
- > the Use Case Tracker MCP provides a shared tool layer
- > Lakebase remains the persistence anchor for tracker data

For executive readers, the main message is that the tracker is not a standalone form app. It is the product-facing part of a governed pattern: Rocketship provides discovery and access, Databricks provides the runtime, MCP provides reusable controlled actions, and Lakebase keeps the source-of-truth data consistent.

2. How to Read This Document

Use this document as the team-facing summary of the Use Case Tracker App. It explains what the app does, how it fits with companion agents, what access model stakeholders should understand, and how Rocketship turns the Databricks-hosted app into an enterprise launch point.

For deeper technical detail, use the earlier documents in the series:

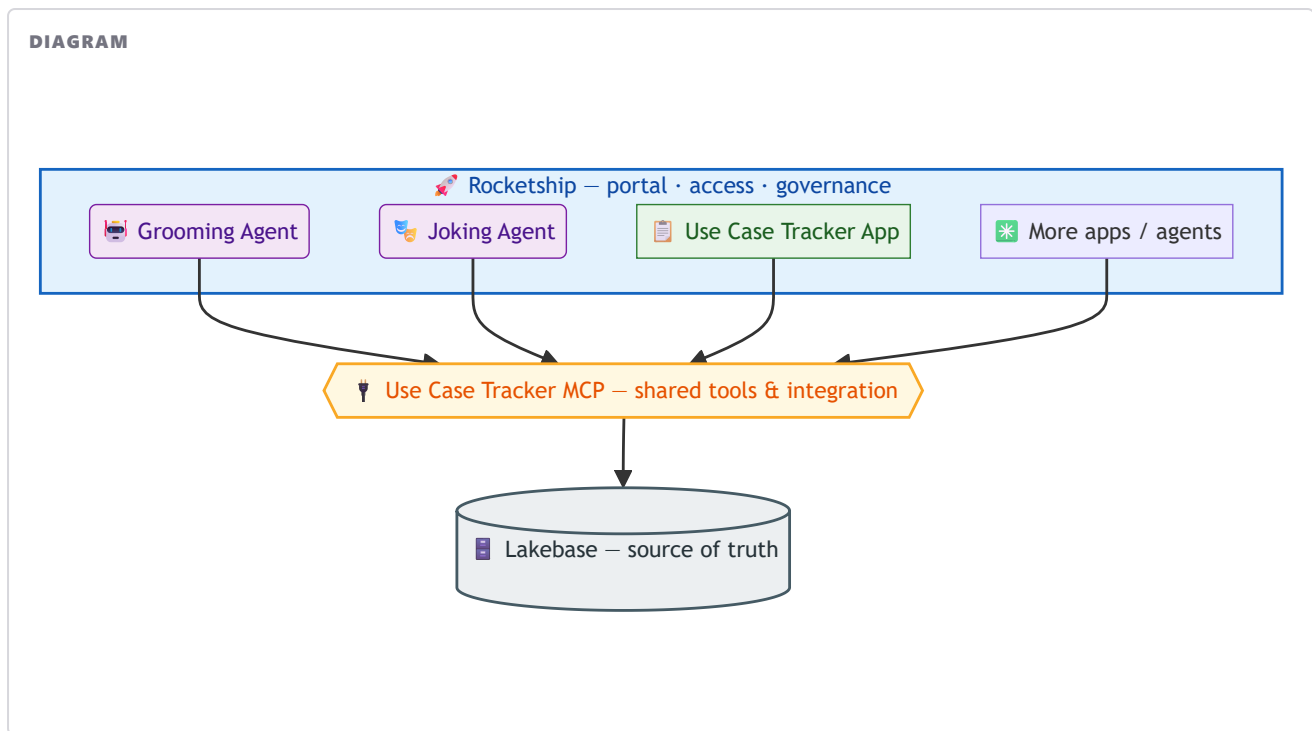
- [MCP Concepts for Governed Apps and Agents](#) explains the reusable MCP concepts.
 - [Secure Databricks Apps + MCP Reference Blueprint](#) explains the generic Databricks architecture.
 - [Use Case Tracker MCP Architecture](#) explains the concrete MCP implementation, tool catalog, permissions, and rollout decisions.
-

3. Ecosystem Architecture

The model follows a clear split: **Rocketship** is where users discover and open governed apps and agents; **Databricks** hosts the running applications. A shared **Use Case Tracker MCP** exposes structured tools so multiple experiences reuse the same tracking logic. **Lakebase** (and related governed stores) holds the persisted use-case data that the tracker and MCP are wired to.

Two figures are included below: **3.1** is the **full ecosystem** (all consumers and extensions). **3.2** zooms in on the **Use Case Tracker App** and **Grooming Agent** only - how each runs as its own Databricks App, why both still call the same MCP, and where Lakebase sits as the system of record.

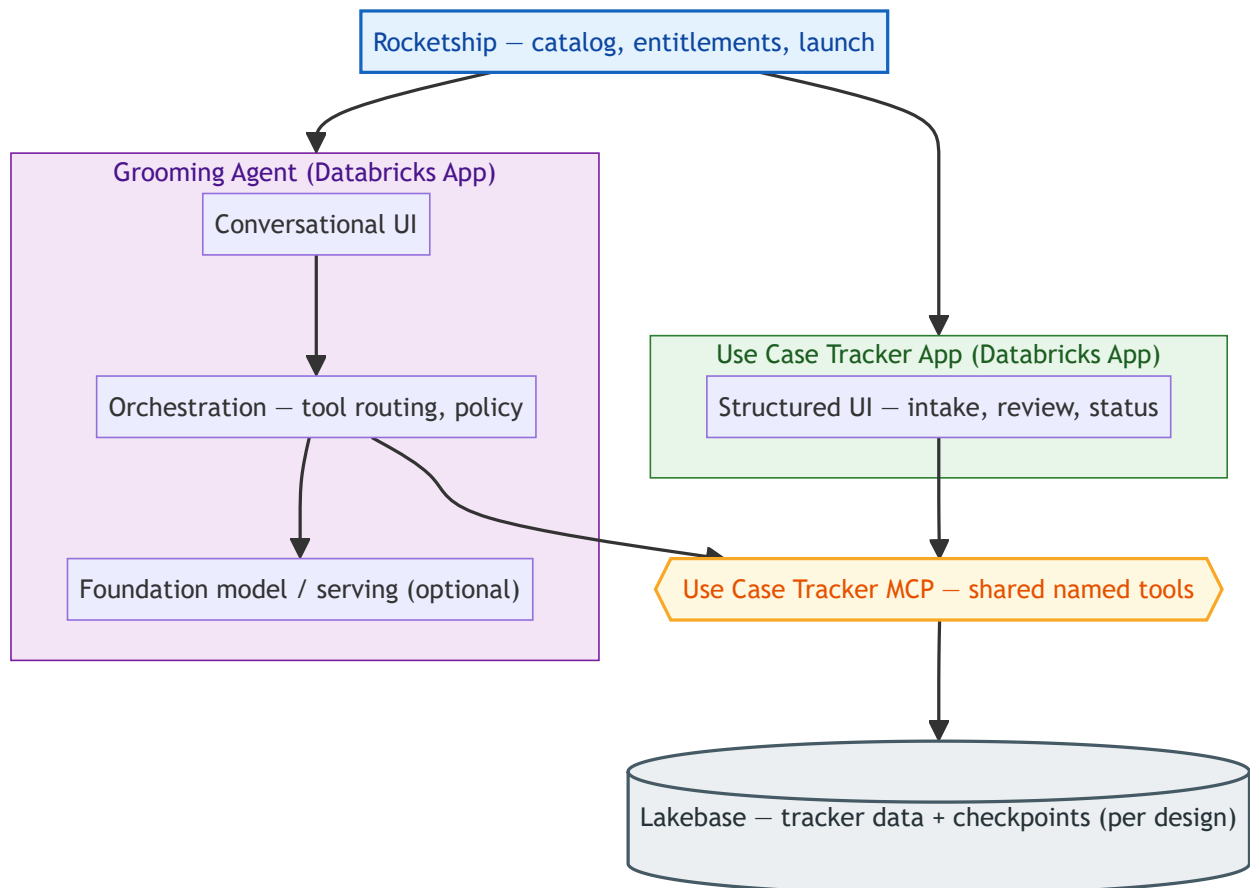
3.1 Platform / ecosystem view (all consumers)



How to read diagram 3.1. People open the **Grooming Agent**, **Joking Agent**, and **Use Case Tracker App** from **Rocketship** (discovery and access). Each consumer calls the **Use Case Tracker MCP** for the same named operations instead of duplicating tracker logic or hitting raw tables. The MCP connects to **Lakebase** (and the broader governed tracker footprint) so use cases, follow-ups, and metadata stay consistent across channels.

3.2 Use Case Tracker App + Grooming Agent (paired view)

Use Case Tracker App is the structured product UI for intake, review, and updates. The **Grooming Agent** is a conversational assistant for triage, gap analysis, and grooming-style recommendations. Both are launched from **Rocketship**, run as separate **Databricks Apps**, and call the **same Use Case Tracker MCP** so tracker behaviour stays one place—not reimplemented in each agent. The Grooming path typically chains **chat / reasoning** (often with a **foundation model** or serving endpoint) to **named MCP tools** (read-focused where policy requires it); the Tracker path emphasises **forms and workflows** that may hold authoritative edits until MCP write tools are hardened.



Icon	Meaning	Shape (Mermaid)
	Rocketship — enterprise catalog, entitlements, launch	Subgraph (platform band)
	AI agents — specialised assistants (grooming vs lighter / demo tone)	Ellipse (...) — typical for autonomous / LLM components
	Use Case Tracker App — structured use-case intake and updates	Rectangle [...] — standard application
	Extension slot — future agents or apps on the same pattern	Rectangle
	MCP — governed tool surface (no ad-hoc SQL from consumers)	Hexagon {{...}} — common for “gateway / API boundary”
	Lakebase — persisted tracker data	Cylinder [(...)] — conventional for databases

How to read diagram 3.2. **Rocketship** is still only discovery and access. The **two boxes** are separate deployed apps: the Tracker favours governed **structured** changes; the Grooming Agent favours **conversation + automation** with the same MCP contract underneath. **Lakebase** remains the persistence anchor; **checkpoints** (where used) are typically via the app **service principal**, separate from end-user OBO on UC business objects - align with your tenant runbook.

Runtime reminder (important for stakeholders). Rocketship is the **access and governance layer**. The apps and MCP **run on Databricks** (for example as Databricks Apps). “Integration with Rocketship” means registering and surfacing those deployed endpoints so the right audiences can launch them with the right entitlements—not moving compute off Databricks.

4. Product Purpose

Use Case Tracker App is the hub’s primary product for capturing, reviewing, and progressing AI and analytics use cases in a structured way: it simplifies intake, review, and updates and shields most users from backend complexity.

Specialised **AI agents** (such as the **Grooming Agent** for triage, gap analysis, and grooming-style recommendations, and the **Joking Agent** for lighter, demo-friendly interactions) sit alongside the app. They are designed to **reuse** the same tracking capability rather than duplicate it, which keeps behaviour consistent as requirements evolve.

The **Use Case Tracker MCP** is the deliberate integration layer between those UIs and agents and the underlying data. It exposes **tool-based** access (no ad-hoc SQL from consumers), which improves reuse, makes automation safer to evolve, and keeps a clearer boundary between “presentation / reasoning” and “system of record.”

Together, the stack supports a modular ecosystem: new agents or apps can be added under Rocketship while still connecting through the same MCP contract, scaling innovation without fragmenting how use cases are stored and interpreted.

5. Access and Permissions

Access is intentionally layered. **Rocketship** governs **who can see and launch** which registered app or agent, aligned with enterprise sharing and catalog conventions. On **Databricks**, each deployed app has its own **service principal** for platform operations (attachments, some background paths), while many user-driven flows rely on **on-behalf-of-user (OBO)** semantics: the signed-in user’s rights are enforced in addition to app configuration.

Users need appropriate **Databricks App** permissions (**CAN USE** or equivalent) on each app they open (tracker UI, Grooming Agent, MCP app if ever exposed directly). Where the app uses a **SQL warehouse**, **Unity Catalog** objects, **Genie** spaces, or other data-plane resources, the same user cohorts need **minimum** warehouse and UC privileges consistent with the tenant design. The **custom MCP** app must be granted to the same groups that depend on it, because agents resolve tracker operations through that app.

Consumer apps (additional Databricks Apps): Callers of the Use Case Tracker MCP need **CAN USE** on the MCP app for the calling principal, and—when **OBO** headers are forwarded—end users need **Lakebase** and **use_case_hub** access as described in [mcp-use-case-tracker/docs/MCP_LAKEBASE_PERMISSIONS.md](#).

For the current operating model emphasising safety on a **shared MCP**, the **live MCP tool surface used by agents is kept read-only** where policy requires it: inspection and recommendations flow through the MCP, while authoritative manual changes may remain in the **Use Case Tracker App** (classic flows) until write tools are explicitly enabled and hardened (identity-aware checks, deny-by-default, audit). **Lakebase** used for agent checkpoints (where applicable) is typically accessed by the **app service principal**, which is separate from end-user OBO grants on business data.

6. Rocketship Integration

Rocketship is integrated as the **enterprise front door: Use Case Tracker App** and agent deployments are **registered** (or planned for registration) so discovery, ownership, and access align with the wider AI Hub / EW portfolio. Internal audiences launch the product and connected agents from Rocketship instead of informal links, which reduces friction and makes permission changes auditable at the portal layer.

Operationally, integration means mapping each Databricks-hosted URL to a **Rocketship catalog entry**, attaching the right **metadata** (purpose, support contact, environment), and ensuring **workspace groups** used for development and pilot users match Rocketship’s entitlements story. The **MCP** remains a **supporting** Databricks App: Rocketship may expose it only to operators, or keep it linked from technical documentation, depending on policy—while still treating it as a first-class dependency in deployment checklists.

Longer term, Rocketship integration supports **scale with control**: the same architecture can add further agents or companion apps without changing how data is persisted, as long as new consumers honour the MCP tool contract and the organisation’s production security bar for OBO, UC, and MCP write policies.

7. Source Notes

This brief reflects the **Use Case Tracker App** architecture as of April 2026. Update URLs, environment names, and permission group labels when promoting across dev / UAT / prod.

PDF: docs/ai-hub-04-tracker-app.pdf - regenerate from this Markdown source with the company document renderer. The PDF should preserve the executive summary, series framing, diagrams, and app access narrative for presentation use.

Generated from Markdown with the EY-Parthenon styled Company Documents Cursor skill.